Un robot de ajedrez que aprende con RL

Propuesta para el trabajo final de Machine Learning (UCU) · Entrega y exposición: jueves 26 de noviembre (clase 31) · Documento para decidir en grupo

La idea en una frase: entrenamos un agente que aprende a jugar al ajedrez por **aprendizaje por refuerzo** (jugando miles de partidas y aprendiendo de los resultados), y lo ponemos dentro de un **robot** con cámara, voz y una cara que muestra lo que "siente": confianza según su chance de ganar y sorpresa según su error de predicción. En la defensa, el tribunal juega contra él.

1. Por qué esta propuesta

- **Cumple la consigna:** marco teórico (RL basado en valor, TD(λ), TD-Leaf, TreeStrap, búsqueda con poda) + caso de aplicación (el agente y el robot). El código ya son scripts de Python (no notebooks), listos para GitHub.
- **Originalidad** (una dimensión de la nota): un robot físico cuyas emociones salen literalmente de las señales del algoritmo es difícil de igualar.
- **Tiene respaldo en la literatura:** el enfoque replica a escala chica trabajos publicados (KnightCap 1998, Giraffe 2015, Meep/TreeStrap de Veness y Silver, NeurIPS 2009). AlphaZero lo descartamos: necesita cómputo que no tenemos.
- **Ya funciona:** no es una idea en el aire; el agente ya aprendió solo en una noche de entrenamiento (ver sección 3).

2. Cómo funciona

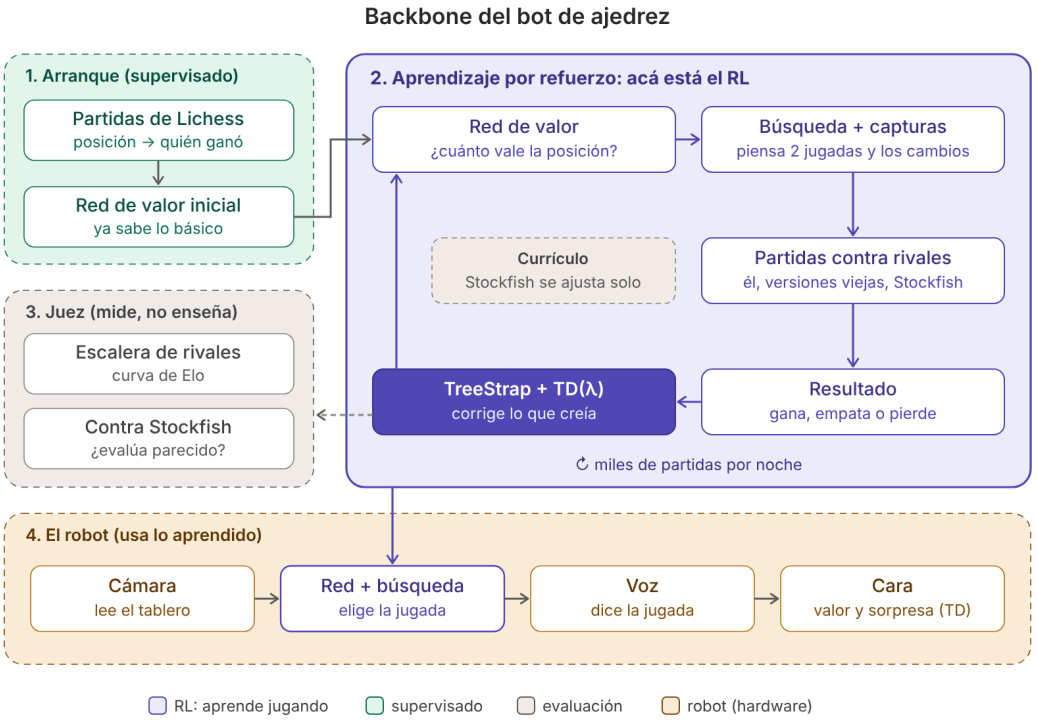


Figura 1. El recuadro violeta es el RL. El arranque es supervisado, el juez solo mide y el robot usa lo aprendido.

- **Arranque:** la red empieza sabiendo lo básico (valor de las piezas o partidas humanas de Lichess), como hicieron KnightCap y Giraffe.

- **RL:** el agente evalúa posiciones con una red neuronal y piensa 1 o 2 jugadas (más los cambios de piezas). Después de cada jugada corrige lo que creía: cada posición que analizó se acerca al valor que mostró la búsqueda (TreeStrap) y el resultado final de la partida viaja hacia atrás (TD(λ)).
- **Rivales:** él mismo, versiones viejas suyas y Stockfish con fuerza limitada que se ajusta solo para que el agente gane más o menos la mitad (currículo).
- **Juez:** escalera de rivales fijos para medir el Elo y comparación con Stockfish. Stockfish mide y sirve de rival, **nunca le enseña** qué jugar.

3. Lo que ya está hecho

Pieza	Estado	Detalle
Arena y medición de Elo	**Hecho**	Rivales fijos (aleatorio, codicioso, minimax de 2 jugadas, Stockfish) y curvas de aprendizaje automáticas.
Agente v1 (TD-Leaf)	**Hecho**	Red de valor + búsqueda de 1 jugada. Entrenado 20.000 partidas desde cero en una noche.
Agente v2	**Hecho y probado**	Búsqueda con capturas, TreeStrap, arranque desde posiciones reales, rivales mezclados, Stockfish con currículo, entrenamiento en paralelo.
Arranque supervisado	**Código listo**	Falta bajar los datos de Lichess y entrenarlo.
Demo 3D interactiva	**Hecho**	Robot y tablero en el navegador: se puede jugar contra él, ver sus emociones y probar altura y ángulo de la cámara.
Revisión de literatura	**Hecho**	Base del marco teórico y justificación de cada decisión.

Primer resultado: el agente aprendió solo

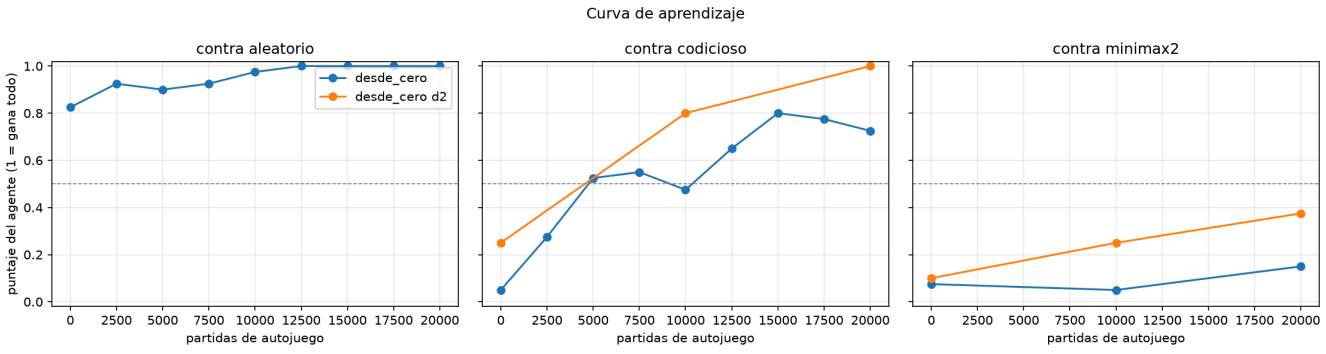


Figura 2. Puntaje del agente (1 = gana todas) a lo largo de 20.000 partidas de autojuego, sin ningún dato humano. Azul: busca 1 jugada; naranja: la misma red buscando 2.

- **Contra el codicioso** (solo come piezas): pasó de ganar el 5 % a ganar el 73 % (con 2 jugadas de búsqueda, el 100 %). Unos **750 puntos de Elo aprendidos solo jugando**.
- **Con la búsqueda fija** (curva naranja), el entrenamiento lo llevó de 0,25 a 1,00 contra el codicioso: la mejora la puso el RL, no la búsqueda.
- **Lo que faltaba:** contra el minimax2 todavía perdía. El diagnóstico (confirmado por la literatura) fue la falta de búsqueda de capturas, arrancar al azar y usar TD-Leaf en vez de TreeStrap. La v2 corrige las tres cosas. En una prueba de control, agregar solo la búsqueda de capturas llevó al mismo evaluador de perder siempre contra el minimax2 a ganarle el 75 %.

4. Lo que falta hacer

Equipo RL (2 personas)

- Bajar datos de Lichess y preentrenar la red
- Correr el bot final (PC, todo el día) y TreeStrap desde cero (laptop, de noche)
- Evaluaciones nocturnas y curvas
- Explicabilidad: valor que aprendió para cada pieza, Grad-CAM sobre el tablero
- Opcional: subir el bot a Lichess

Equipo robot (eléctrica + 1)

- Diseño en Fusion con las medidas del tablero
- Impresión 3D de cabeza y base
- Visión: detectar vacía, blanca o negra por casilla
- Pantalla con caras y voz
- Integración con el agente

Todos: informe (marco teórico + caso + resultados), README del repositorio, diapositivas, ensayo de la demo y video de respaldo.

5. Decisiones de diseño que tenemos que tomar

¿Nos copa el proyecto? Y elegir ya el tema "aprendizaje por refuerzo" en la herramienta de la webasignatura (la consigna pide hacerlo antes).

Arranque del bot final: (a) solo valor del material, 100 % sin datos humanos, o (b) partidas humanas de Lichess, más fuerte. En los dos casos el RL se hace encima. Propuesta: (b), y mantener la corrida desde cero como comparación.

Pantalla de la cara: (a) un celular viejo mostrando una página, gratis e inmediato; (b) dos pantallas redondas como ojos + ESP32, más lindo y más trabajo; (c) pantalla de 7" HDMI, las baratas tardan unos 15 días en llegar.

Visión: detectar por casilla solo "vacía / blanca / negra" y deducir la jugada comparando con las jugadas legales (más simple y robusto), o clasificar las 13 clases de piezas (más difícil). Propuesta: la primera.

Cuerpo: cabeza y base impresas en 3D + mástil de caño de PVC (un mástil impreso de 45 a 50 cm sale débil). ¿Quién tiene impresora o acceso a una? Según la demo, la cámara tiene que quedar a unos 45 a 50 cm sobre el tablero para verlo entero.

Extras: ¿servo para que asienta con la cabeza? ¿bot público en Lichess? Suman en la defensa, pero solo si sobra tiempo.

6. Qué comprar

Componente	Para qué	Precio aprox. (MercadoLibre UY)	¿Necesario?
Aro de luz LED USB (16 cm, con pinza)	Luz pareja: lo que más estabiliza la visión	unos $190 (llega en 1 día)	Sí
Webcam USB (Logitech C270 o similar)	Leer el tablero	US$ 38 a 44	Solo si nadie tiene
Caño de PVC 32 a 40 mm + filamento PLA	Mástil y piezas impresas	ferretería / quien imprima	Sí
Parlante USB chico	Voz del robot	unos $490	Opcional (sirve la laptop)
Servo SG90	Asentir con la cabeza	unos $220	Opcional
Pantalla redonda 1,28" + ESP32	Ojos (opción b)	US$ 35 por ojo + $750	Solo si elegimos (b)

Kit básico: unos $900 más la webcam si hace falta. Precios vistos el 12/11; evitar publicaciones con entrega "15 días después de tu compra" porque llegan tarde.

7. Cronograma (13 al 26 de noviembre)

Fechas	RL	Robot
13 y 14	Datos de Lichess, preentrenamiento, lanzar corridas v2	Medidas, compras, diseño en Fusion, caras
15 al 18	Corridas continuas, evaluaciones nocturnas	Impresión 3D, visión con fotos del tablero
19 al 22	Ajustes, versión final del agente	Pantalla, voz, integración. **22: todo conectado**
23 y 24	Curvas finales, explicabilidad, informe	
25 y 26	Ensayo de la demo, video de respaldo, entrega y exposición	

8. Riesgos y plan B

- **Lo más riesgoso es la integración** (cámara + agente + cara + voz en vivo), no el RL. Por eso el 22 tiene que estar todo conectado, aunque sea feo.
- **Si nos atrasamos**, se recorta en este orden: voz → cuerpo 3D (soporte de cartón) → cámara (tablero en pantalla). La curva de aprendizaje y los experimentos de RL no se tocan: es lo que se evalúa.
- **Si la demo en vivo falla:** video grabado + la demo 3D en el navegador.

Referencias principales: Baxter, Tridgell y Weaver (1998), KnightCap / TD-Leaf(λ). Lai (2015), Giraffe. Veness, Silver, Uther y Blair (2009), Bootstrapping from Game Tree Search. Silver et al. (2017), AlphaZero. McIlroy-Young et al. (2020), Maia. Base de datos abierta de Lichess.